

# Семинар 2 · Проектируем схему своего объекта

Базы знаний и базы данных · кафедра ИУ-1 · 3 курс · неделя 2 · 90 минут

|             |                                                                                                                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Цель        | Спроектировать схему данных для предметной области вашего проекта на трёх уровнях — диаграмма «сущность — связь», DDL, миграция в PostgreSQL — и провести взаимную проверку схем по контрольному списку.             |
| Чек-поинт 2 | В репозитории команды: seminar02/schema.mmd (диаграмма «сущность — связь»), seminar02/migrations/V1__init.sql и V2__*.sql, seminar02/review.md с замечаниями от другой команды. Работает <code>make migrate</code> . |
| Баллы       | 2 балла. В срок — 2, позже — 1. Работа командная, один репозиторий на команду; вклад каждого виден по коммитам.                                                                                                      |
| Что нужно   | Репозиторий курса с поднятым стеком (семинар 1), команда 2–3 человека, выбранная предметная область.                                                                                                                 |

## Как устроен семинар

Пять частей. Первые три — проектирование снизу вверх по уровням из лекции: сущности → ER → DDL. Четвёртая — ревью чужой схемы, пятая — исправление своей. Схема, которую вы сделаете сегодня, — фундамент проекта до конца семестра, поэтому лучше потратить лишние десять минут на сущности, чем переделывать в ноябре. Автопроверки нет: качество схемы проверяется глазами — вашими, другой команды и моими на семинаре 3.

## Часть 1 · Сущности и связи словами (15 мин)

Не открывая редактор, выпишите в seminar02/README.md:

- **8–12 сущностей** вашей предметной области. Тест: сущность — это то, о чём можно спросить «сколько их?» и «какой именно?». Инженер — сущность; «ремонт сделал Иванов» — нет, это связь.
- Для каждой — **естественный ключ**, если он есть (код агрегата, серийный номер, табельный номер), и 3–5 атрибутов.
- **Связи** между сущностями в форме «A — B: 1:N, обязательна ли, есть ли атрибуты у связи». Например: «ремонт — запчасть: M:N, атрибут связи — количество».
- **Что меняется во времени** и чью историю спросят: размещение оборудования, калибровка, пороги, ответственные.

## Минимальный набор для любого объекта

Что бы вы ни выбрали, в схеме должны быть: **объект наблюдения** (агрегат, дрон, помещение, пациент...), его **иерархия или размещение** (площадка / здание / отделение), **источники сигналов** (датчики, каналы), **телеметрия**, **события** с уровнем серьёзности, **воздействия** (ремонт, обслуживание, вмешательство) с исполнителем и расходуемыми ресурсами, **документы** (пока — только ссылка и тип). Остальное — специфика вашей области.

## Часть 2 · Диаграмма «сущность — связь» в Mermaid (20 мин)

Диаграмма — в файле `seminar02/schema.mmd`. GitHub и GitLab рендерят Mermaid прямо в README, так что вставьте её туда же в блок ````mermaid`. Пример из лекции (расширенная станция) лежит в `seminar02/example/schema.mmd` — не копируйте, но подсмотреть синтаксис можно.

```
erDiagram
    station ||--o{ unit : "размещает"
    unit ||--o{ sensor : "несёт"
    sensor ||--o{ telemetry : "измеряет"
    unit ||--o{ maintenance : "обслуживается"
    engineer ||--o{ maintenance : "выполняет"
    maintenance }o--o{ part : "расходует"

    unit {
        int id PK
        text code UK "K-2"
        int station_id FK
        text model
    }
    maintenance_part {
        int maintenance_id PK, FK
        int part_id PK, FK
        int qty
    }
```

Обозначения: `||` — ровно один, `o{` — ноль или много, `|{` — один или много. M:N на диаграмме можно показать напрямую `{o--o{}`, но в DDL это всегда таблица связи.

Что проверить, глядя на диаграмму:

- у каждой сущности есть ключ; у справочников (тип датчика, уровень серьёзности, инженер) — тоже;
- нет сущностей-«островов» без связей и нет связей «всё со всем»;
- M:N разрешены через таблицу связи; атрибуты связи (количество, роль, дата) лежат в таблице связи;
- то, что меняется во времени, имеет либо `valid_from` / `valid_to`, либо отдельную таблицу истории.

## Часть 3 · DDL и первая миграция (25 мин)

DDL — в `seminar02/migrations/V1__init.sql`. Все таблицы проекта создаются в схеме `project`, чтобы не мешать учебному датасету в `public`. Шаблон файла уже лежит в репозитории.

```
create schema if not exists project;
set search_path = project;

create table station (
    id          int generated always as identity primary key,
    code        text not null unique,
    name        text not null,
    region      text not null
);

create table unit (
    id          int generated always as identity primary key,
    code        text not null unique,           -- естественный ключ: 'K-2'
    station_id  int not null references station(id),
    kind        text not null references unit_kind(code),
    model       text not null,
    commissioned date not null,
    created_at  timestampz not null default now()
);
```

Применить и проверить:

```
make migrate      # применяет seminar02/migrations/V*.sql по порядку; повторно — только новые
make psql         # \dt project.* — таблицы на месте?
```

Требования к V1:

- суррогатный первичный ключ (identity) + UNIQUE на естественный там, где он есть;
- все внешние ключи объявлены; для таблиц связи — составной первичный ключ;
- NOT NULL везде, где значение обязательно; хотя бы два CHECK (диапазон, положительное количество, порядок дат);
- перечисления — справочник или CHECK, но не свободный text;
- время — только timestamptz; измерения — numeric, не float;
- comment on table / column для всего, что не очевидно по имени — минимум по одному на таблицу;
- в конце файла — 3-5 INSERT с тестовыми данными, чтобы схему можно было «пощупать».

#### Если миграция не применяется

- **ошибка в SQL** — make migrate останавливается на первой ошибке и печатает её; файл не считается применённым, правьте и запускайте снова;
- **таблица уже существует** — вы правили V1 после применения. На семинаре это простительно: make migrate -reset снесёт схему project и применит всё заново. В проекте так делать нельзя — только новая миграция;
- **порядок таблиц** — таблица со ссылкой создаётся после той, на которую ссылается. Справочники — первыми.

## Часть 4 · Взаимная проверка схем (15 мин)

Команды объединяются в пары (назначу на семинаре). Каждая команда читает README, диаграмму и V1 другой и заполняет для неё seminar02/review.md — файл кладётся в репозиторий *проверяемой* команды через pull request или просто присылается им в чат. Шаблон с контрольным списком — в репозитории; кратко он выглядит так:

| Раздел                  | Что смотрим                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Сущности                | все ли существительные предметной области стали таблицами; нет ли «инженер — text»; M:N через таблицу связи         |
| Ключи                   | PK + UNIQUE на естественный; все внешние ключи есть; составной первичный ключ у таблиц связи; ON DELETE осмысленный |
| Ограничения целостности | NOT NULL; перечисления не text; CHECK на диапазоны                                                                  |
| Время                   | timestamptz; история для того, что меняется                                                                         |
| Нормализация            | нет списков в строке, нет полей sensor_1, sensor_2; нет транзитивных зависимостей                                   |
| Читаемость              | имена таблиц в единственном числе и одном стиле; есть comment on; V1 применяется с первого раза                     |

Формат замечания: *что не так* → *почему это проблема* → *как исправить*. Три содержательных замечания лучше десяти «переименуйте столбец». Похвалить за удачное решение тоже можно — это тоже обратная связь.

## Часть 5 · V2 по замечаниям (15 мин)

Прочитайте замечания к своей схеме. С чем согласны — исправьте **второй миграцией** V2\_\_<что\_сделали>.sql, не редактируя V1 (V1 уже «применён в проде» — правило из лекции). С чем не согласны — напишите почему, в ответ в review.md.

```
-- V2__engineer_lookup.sql
set search_path = project;
create table engineer (
    id          int generated always as identity primary key,
    full_name   text not null,
    tab_no      text not null unique
);
alter table maintenance add column engineer_id int references engineer(id);
-- перенос данных из текстового столбца, затем:
-- alter table maintenance alter column engineer_id set not null;
-- alter table maintenance drop column engineer;
```

Если у вас ADD + дозаполнение + NOT NULL + DROP в одной миграции — это нормально для семинара. На живой системе это четыре шага «расширение — сжатие» (expand/contract) — вспомните слайд.

## Дома · Упражнения по нормализации

Три задачи в seminar02/exercises.md — на бумаге или в том же файле. Не оцениваются, но разберём на семинаре 3, и похожие будут в письменной работе на неделе 8.

| № | Задача                                                                                                                                                                                                                                                   |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Дана таблица flight(flight_no, date, aircraft, aircraft_seats, captain, captain_licence, route_from, route_to, route_distance). Выпишите функциональные зависимости, найдите ключ, приведите к 3NF.                                                      |
| 2 | Таблица reading(sensor_tag, unit_code, station_name, ts, value, unit_of_measure) — 10 <sup>9</sup> строк. Какие аномалии возникнут при переезде агрегата? Что нужно вынести и какую денормализацию вы бы всё же оставили ради скорости чтения, и почему? |
| 3 | Спроектируйте хранение <b>порогов срабатывания</b> для датчиков так, чтобы можно было ответить: «какой порог действовал для TE-302 12 марта 2025?» и «кто и когда его изменил?». DDL с ограничением на непересечение интервалов.                         |

## Что сдаётся и как оценивается

| Критерий          | Проверяется                                                                                                 | Баллы |
|-------------------|-------------------------------------------------------------------------------------------------------------|-------|
| Схема применяется | make migrate-reset из чистого клона проходит без ошибок; таблицы в схеме project; есть тестовые INSERT      | 0.5   |
| Качество V1       | контрольный список: ключи, внешние ключи, NOT NULL, CHECK, справочники вместо text, timestampz, комментарии | 1.0   |
| Проверка и V2     | содержательный review.md для другой команды; V2 закрывает хотя бы два замечания из полученного ревью        | 0.5   |
| Итого             |                                                                                                             | 2.0   |

### **К следующей неделе**

- Чек-поинт 2 в репозитории команды: README с сущностями, schema.mmd, V1, V2, review.md.
- Упражнения по нормализации — хотя бы попытка по каждой.
- Прочитать: Клеппман, гл. 2 «Модели данных и языки запросов».
- Неделя 3 — продвинутый SQL на учебном датасете: оконные функции, CTE, рекурсия. Освежите GROUP BY, HAVING, подзапросы.